

Bài 6: CÂY NHỊ PHÂN

Mục tiêu:

Về kiến thức:

+ Nắm vững các khái niệm cơ bản về mô hình Cây nhị phân, các phương pháp biểu diễn Cây nhị phân.

Về kỹ năng:

+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên cây nhị phân trong các bài toán cụ thể.

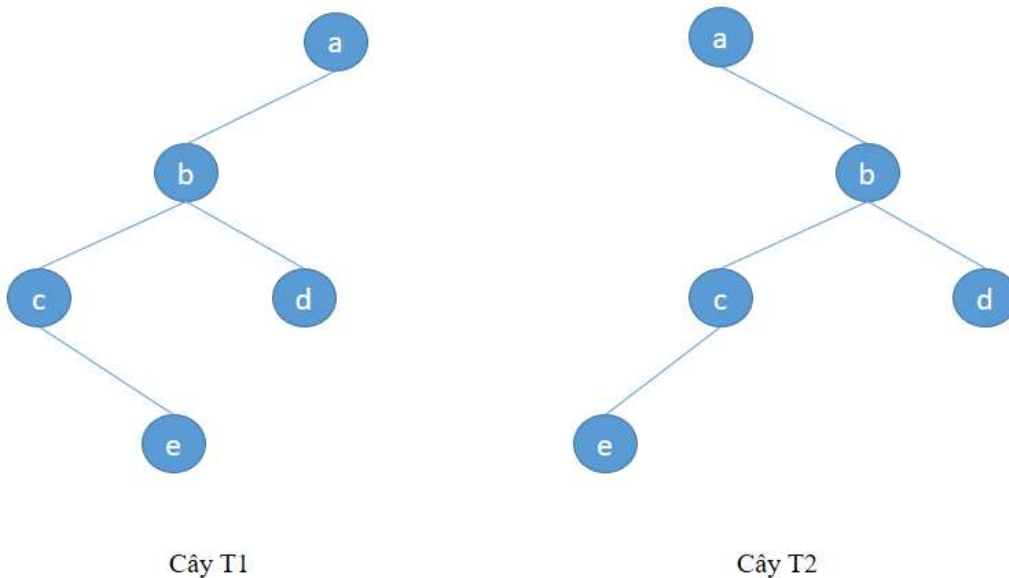
6.1 Định nghĩa cây nhị phân và phân loại cây nhị phân

6.1.1 Định nghĩa cây nhị phân

Cây nhị phân là cây mà mỗi nút có tối đa hai con.

Vì mỗi nút có không quá hai con, nên ta sẽ gọi hai con tương ứng là *con trái* và *con phải*. Như vậy, các nút con của cây được phân biệt thứ tự rõ ràng.

Ví dụ 1 (Hình 6.1): ví dụ về cây nhị phân có số nút như nhau nhưng thứ tự khác nhau



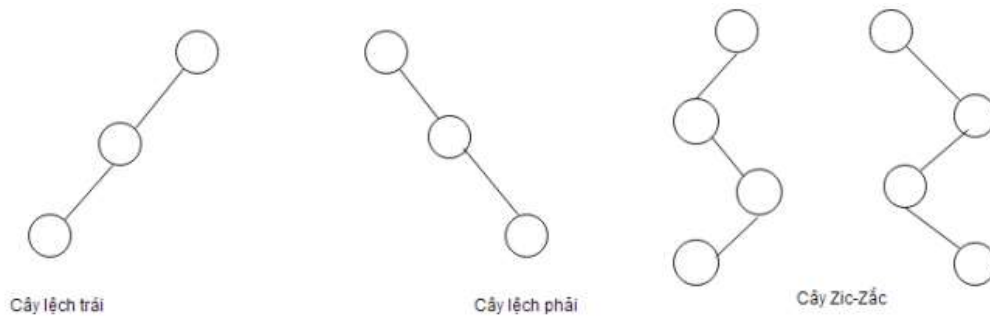
Hình 6.1: Ví dụ về cây nhị phân

Tính chất của cây nhị phân:

- Số đỉnh lớn nhất ở mức i của cây nhị phân là 2^{i-1} , $i \geq 1$
- Cây nhị phân với chiều cao là k có không quá $2^k - 1$ nút, $k \geq 1$
- Một cây nhị phân có n nút thì có chiều cao tối thiểu là $\log_2(n+1)$

6.1.2 Các loại cây nhị phân

- Cây suy biến: là loại cây lệch trái, cây lệch phải hoặc cây zig-zắc



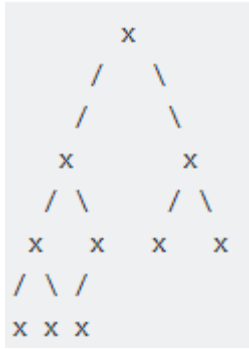
Hình 6.2: Cây nhị phân suy biến

- Cây nhị phân hoàn hảo: là cây nhị phân thỏa mãn:
 - + Mọi nút lá đều có cùng một mức (độ sâu)
 - + Các nút trong có đủ hai con



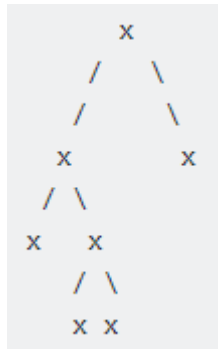
Hình 6.3: Cây hoàn hảo

- Cây hoàn chỉnh: là cây nhị phân có chiều cao là n thỏa mãn
 - + Nếu không tính mức n thì nó là cây hoàn hảo
 - + Các nút có độ sâu là n thì bị lệch trái nhất có thể



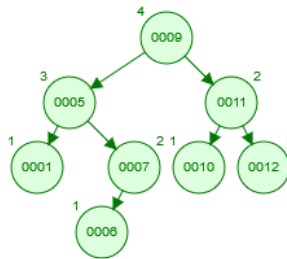
Hình 6.4: Cây hoàn chỉnh

- Cây nhị phân đầy đủ: là cây nhị phân mà mọi nút trong đều có đủ hai con (trừ nút lá)



Hình 6.5: Cây đầy đủ

- Cây cân bằng (AVL): là cây mà tại mọi nút chiều cao của cây con trái và cây con phải lệch nhau không quá 1 đơn vị



Hình 6.6: Cây cân bằng (AVL)

Nhận xét: Trong các cây nhị phân có cùng số đỉnh thì cây nhị phân suy biến có chiều cao lớn nhất và cây nhị phân hoàn hảo có chiều cao thấp nhất. Trong cây hoàn hảo, số đỉnh ở mức thứ i là 2^i nút, số nút trên cây có chiều cao là n là $2^{n+1}-1$ nút.

6.2 Biểu diễn cây nhị phân trên máy tính

Có hai phương pháp biểu diễn cây nhị phân

- Biểu diễn sử dụng mảng
- Biểu diễn sử dụng con trỏ

6.2.1 Biểu diễn cây nhị phân sử dụng mảng

Tương tự với cách biểu diễn cây tổng quát. Tuy nhiên, cây nhị phân chỉ có tối đa hai con nên mỗi nút (đỉnh) là một cấu trúc gồm 3 trường: *data*, *left*, *right*

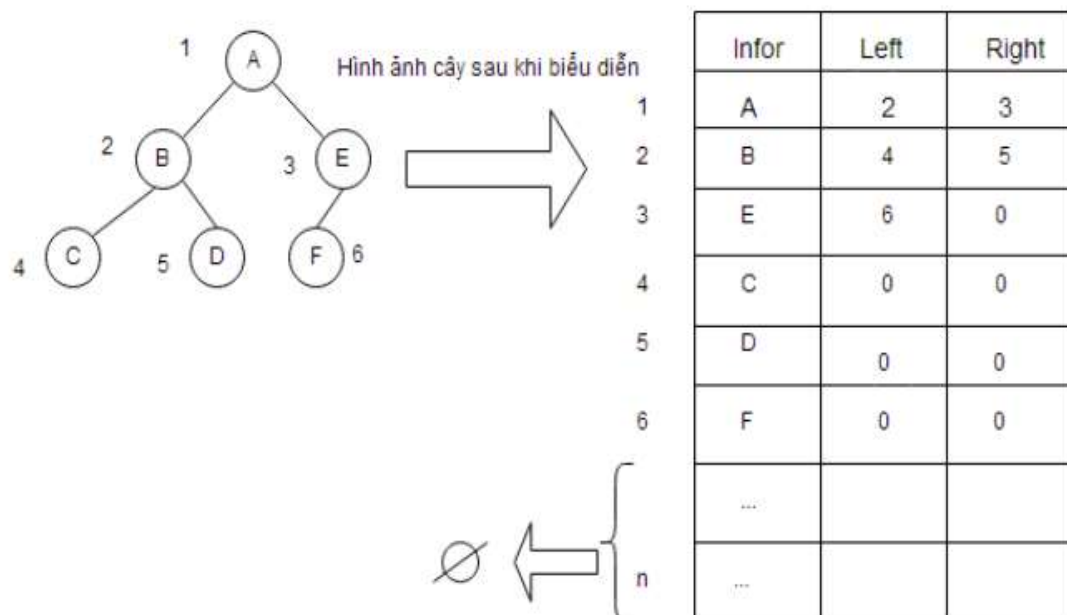
data: là dữ liệu của đỉnh

left: con trái của đỉnh

right: con phải của đỉnh

Với cách biểu diễn bằng mảng thì cây sau khi biểu diễn được xem như là mảng các đỉnh. Khi đó, các thao tác trên cây bản chất là thao tác trên mảng.

Ví dụ: biểu diễn cây bằng cấu trúc mảng



Hình 6.7: Ví dụ biểu diễn cây nhị phân bằng mảng

Chú ý: Việc đánh số thứ tự các nút có thể đánh từ 0, 1, Trong trường hợp các nút

con trái và phải rỗng có thể đặt 0 hoặc -1 tùy vào việc chọn thứ tự đánh các nút

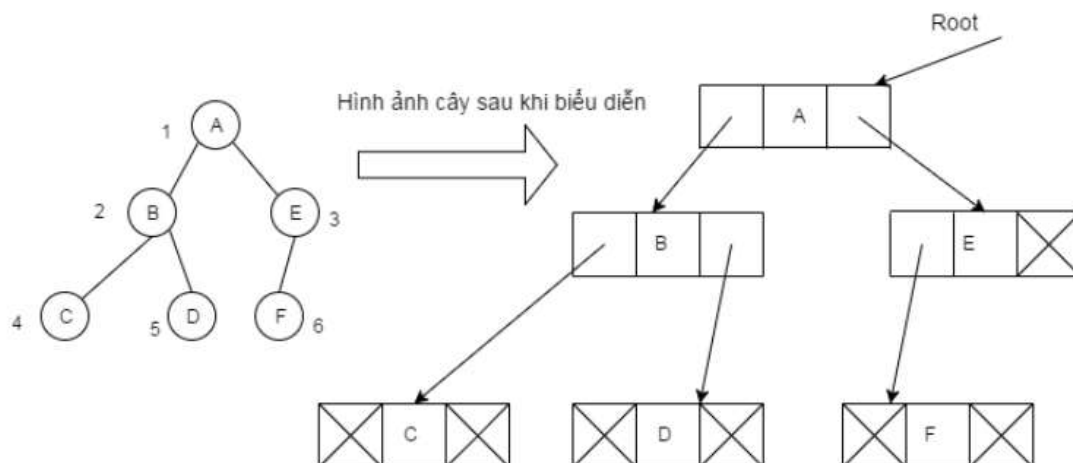
Cú pháp biểu diễn cây bằng mảng:

```
#include<stdio.h>
#define N 100
// định nghĩa một Node trên cây
typedef struct Node
{
    item data;
    int left,right;
}Node;
// cây nhị phân ban đầu là một mảng các Node
typedef struct bTree1
{
    Node elems[N];
    int size;
}bTree1;
```

6.2.2 Biểu diễn cây nhị phân sử dụng con trỏ

Với cách biểu diễn này mỗi nút trên cây vẫn có 3 trường: *data*, *left* và *right*. Trong đó trường *left*, *right* có kiểu con trỏ. Để quản lý cây, chúng ta sử dụng biến con trỏ trỏ tới gốc của cây. Để truy cập tới các đỉnh khác trên cây phải xuất phát từ gốc của cây.

Ví dụ: Biểu diễn cây nhị phân sử dụng con trỏ



Hình 6.8: Ví dụ biểu diễn cây nhị phân sử dụng con trỏ

Cú pháp biểu diễn:

```
#include<stdio.h>
// định nghĩa một Node trên cây
typedef struct Node
{
    item data;
    Node *left,*right;
}Node;
Node *bTree2;
```

6.3 Các thao tác trên cây nhị phân

Đối với cây nhị phân cài đặt sử dụng mảng coi như bài tập. Trong bài học này trình bày các thao tác trên cây nhị phân sử dụng cấu trúc cài đặt cây nhị phân sử dụng con trỏ. Các phép toán lần lượt như sau:

1- Khởi tạo cây: là phép toán tạo ra cây rỗng. Cây rỗng là cây không có chứa nút(đỉnh) nào. Như vậy con trỏ quản lý cây sẽ trỏ tới giá trị NULL. Đoạn mã chương trình như sau:

```
void makeNullTree(bTree2 *T)
{
    (*T)=NULL;
}
```

2- Kiểm tra cây rỗng: Để kiểm tra cây rỗng hay không ta kiểm tra trên cây có nút nào không. Trong trường hợp cây được cài đặt bằng con trỏ nên chúng ta chỉ cần kiểm tra con trỏ T(T là biến quản lý cây) có trỏ tới NULL hay không. Nếu cây rỗng thì hàm kiểm tra nhận giá trị là true, ngược lại nhận giá trị false. Đoạn mã chương trình như sau:

```
int emptyTree(bTree2 T)
{
    return T==NULL;
}
```

3- Xác định con trái, con phải của một nút trên cây nhị phân

Để tìm con trái (con phải) của một nút trên cây, chúng ta truy cập vào trường dữ liệu left(right) tương ứng.

Đoạn mã tìm con trái như sau:

```
bTree2 leftChild(bTree2 T)
{
    if (T!=NULL) return T->left;
    else return NULL;
}
```

Đoạn mã tìm con phải của một nút trên cây:

```
bTree2 rightChild(bTree2 T)
{
    if (T!=NULL) return n->right;
    else return NULL;
}
```

4- Kiểm tra nút lá trên cây: Một nút là nút lá nếu con trái và con phải rỗng tức là con trái và con phải đều là NULL

Đoạn mã kiểm tra nút lá:

```
int isLeaf(bTree2 T)
{
    if(T!=NULL)
        return(leftChild(T)==NULL)&&(rightChild(T)==NULL);
    else return NULL;
}
```

5- Đếm số nút trên cây

Nếu cây khác rỗng thì số nút trên cây=1+ số nút trên cây con trái+số nút trên cây con phải. Đoạn mã chương trình viết dưới dạng đệ quy như sau:

```
int numberNodes (bTree2 T)
{
    if(emptyTree(T)) return 0;
    else return 1+ numberNodes (leftChild(T))+ numberNodes (rightChild(T));
}
```

6- Tìm nút có nội dung là x trên cây nhị phân. Nếu cây khác rỗng:

- So sánh gốc của cây với x:

- + Nếu gốc có nội dung =x thì kết luận gốc là nút cần tìm
- + Nếu gốc có nội dung <> x thì tìm kiếm trên cây con trái và cây con phải của cây.

Đoạn mã viết đệ quy như sau:

```
bTree2 search (bTree2 tree, int x)
{
    bTree2 p;
    if (tree->data == x)
        return tree;
    if (tree == NULL)

        return NULL;
    p = search(tree->left, x);
    if (p == NULL)
        search(tree->right, x);
}
```

7- Phép toán duyệt cây. Tương tự như cây tổng quát, cây nhị phân cũng có 3 cách duyệt: duyệt trước, duyệt giữa và duyệt sau. Vì cây nhị phân chỉ có tối đa là hai con nên cách duyệt cũng đơn giản hơn so với cây tổng quát.

Duyệt cây theo thứ tự trước (NLR)

- Nguyên tắc duyệt theo thứ tự trước: Duyệt gốc trước, duyệt cây con trái theo thứ tự trước, duyệt cây con phải theo thứ tự trước.

- Các bước thực hiện: Nếu cây khác rỗng thì:

- + duyệt gốc
- + duyệt cây con trái theo thứ tự trước
- + duyệt cây con phải theo thứ tự trước

- Chương trình:

```
void preOrder(bTree2 * r)
{
    if (r != NULL)
    {
        printf("%c", r->data);
        preOrder(r->leftChild);
        preOrder(r->rightChild);
    }
}
```

Duyệt cây theo thứ tự giữa (LNR)

- Nguyên tắc duyệt: Duyệt con trái theo thứ tự giữa, duyệt gốc và duyệt con phải theo thứ tự giữa

- Các bước thực hiện: Nếu cây khác rỗng:

+ Duyệt con trái theo thứ tự giữa

+ Duyệt gốc

+ Duyệt con phải theo thứ tự giữa

- Chương trình:

```
void inOrder(bTree2 * r)
{
    if (r != NULL)
    {
        inOrder (r->leftChild);
        printf("%c", r->data);
        inOrder (r->rightChild);
    }
}
```

Duyệt cây theo thứ tự sau (LRN)

- Nguyên tắc duyệt: Duyệt con trái và con phải theo thứ tự sau, cuối cùng duyệt gốc

- Các bước thực hiện: Nếu cây khác rỗng:

+ Duyệt con trái theo thứ tự sau

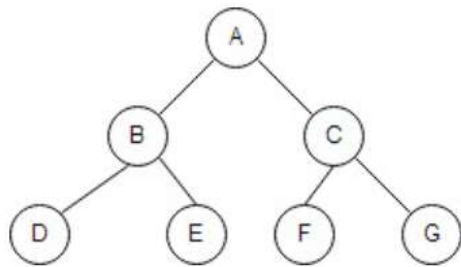
+ Duyệt con phải theo thứ tự sau

+ Duyệt gốc

- Chương trình:

```
void postOrder(bTree2 * r)
{
    if (r != NULL)
    {
        postOrder (r->leftChild);
        postOrder (r->rightChild);
        printf("%c", r->data);
    }
}
```

Ví dụ: Duyệt cây sau theo thứ tự trước, giữa, sau:



Kết quả:

Duyệt theo thứ tự trước: *A BDE CFG*

Duyệt theo thứ tự giữa: *DBE A FCG*

Duyệt theo thứ tự sau: *DEB FGC A*

Hình 6.9: Ví dụ duyệt cây theo thứ tự trước, giữa, sau

6.4 Thực hành

6.4.1 Mục tiêu

Bài thực hành giúp sinh viên biết cách cài đặt cây nhị phân theo hai cách : bằng mảng và con trỏ. Sau đó, sinh viên thực hành thành thạo trên các phép toán của cây nhị phân. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng

6.4.2 Bài thực hành mẫu

Bài 1: Cài đặt cây nhị phân sử dụng mảng có khóa là các số nguyên. Viết chương trình nhập và in cây nhị phân theo thứ tự trước, giữa, sau.

```
#include <stdio.h>
#include <conio.h>
#define max 100
#define MAXLENGTH 100      //chi so toi da cua mang
#define NIL -1
typedef int DataType;
```

```

typedef int Node;
typedef struct {
    DataType Data[MAXLENGTH]; //Luu gia tri cua nut
    int MaxNode;
}Tree;
//Kiem tra cay rong
int EmptyTree(Tree T) {
    return T.MaxNode == 0;
}
//Xac dinh nut goc trong cay
Node Root(Tree T) {
    if (!EmptyTree(T))
        return 0;
    else
        return NIL;
}
//con trai cua nut p
int Left_Child(Node p, Tree T) {
    return 2*(p+1) - 1;
}
//con phai cua nut p
int Right_Child(Node p, Tree T) {
    return 2*(p+1);
}
//duyet tien tu
void NLR(Node p, Tree T) {
    if(T.Data[p]!=NIL) {
        printf("%d ", T.Data[p]);
        NLR(Left_Child(p,T),T);
        NLR(Right_Child(p,T),T);
    }
}
//duyet trung tu
void LNR(Node p, Tree T) {
    if(T.Data[p]!=NIL) {
        LNR(Left_Child(p,T),T);
        printf("%d ", T.Data[p]);
        LNR(Right_Child(p,T),T);
    }
}
//duyet hau tu
void LRN(Node p, Tree T) {
    if(T.Data[p]!=NIL) {

```

```

        LRN(Left_Child(p,T),T);
        LRN(Right_Child(p,T),T);
        printf("%d ", T.Data[p]);
    }
}
/*doc cay
neu khong co con trai hoac phai thi nhap -1*/
void Read_Tree(Tree * T) {
    int i = 0, Child = 0;
    printf("Nhap vao so nut: ");
    scanf("%d",&(*T).MaxNode);
    while(i<(*T).MaxNode) {
        if(i==0)
        {
            printf("Nut goc:");
            scanf("%d",&(*T).Data[i]);
            (*T).Data[Left_Child(i,*T)] = NIL;
            (*T).Data[Right_Child(i,*T)] = NIL;
            i++;
        }
        else
        if((*T).Data[Child]!=NIL)
        {
            int k;
            printf("Con trai %d: ",Child);
            k = Left_Child(Child,*T);
            scanf("%d",&(*T).Data[k]);
            if((*T).Data[k] != NIL) {
                (*T).Data[Left_Child(k,*T)] = NIL;
                (*T).Data[Right_Child(k,*T)] = NIL;
                i++;
            }
            printf("Con phai %d: ",Child);
            k = Right_Child(Child,*T);
            scanf("%d",&(*T).Data[k]);
            if((*T).Data[k] != NIL) {
                (*T).Data[Left_Child(k,*T)] = NIL;
                (*T).Data[Right_Child(k,*T)] = NIL;
                i++;
            }
            Child++;
        }
    }
}

```

```

int main() {
    Tree T;
    //clrscr();
    Read_Tree(&T);
    printf("Duyet tien tu.\n");
    NLR(Root(T),T);
    printf("\nDuyet trung tu.\n");
    LNR(Root(T),T);
    printf("\nDuyet hau tu.\n");
    LRN(Root(T),T);
    getch();
    return 1;
}

```

6.4.3 Bài tập luyện tập

Bài 2: Cài đặt cây nhị phân sử dụng con trỏ có khóa là các số nguyên. Viết các phép toán trên cây nhị phân. Viết chương trình nhập một cây nhị phân có khóa là số nguyên dương (điều kiện dừng khi nhập vào số nguyên âm) và in cây nhị phân vừa nhập theo thứ tự trước, giữa, sau.

Bài 3: Cài đặt cây nhị phân sử dụng con trỏ có dữ liệu là xâu kí tự. Viết hàm tìm độ sâu(chiều cao) của cây. Viết chương trình nhập, in cây vừa nhập và cho biết cây vừa nhập có độ cao bằng bao nhiêu.

Bài 4: Cài đặt cây nhị phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm tính số lượng nút lá trên cây nhị phân

Bài 5: Cài đặt cây nhị phân sử dụng con trỏ có dữ liệu ch

ứa kiểu số nguyên. Viết hàm kiểm tra xem một cây có phải là cân bằng hay không. Cây cân bằng là cây mà chiều cao của cây con trái và con phải lệch nhau không quá 1.

Bài 6: Cài đặt cây nhị phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm kiểm tra xem một cây có phải là cân bằng hay không. Viết hàm tìm giá trị chẵn lớn nhất trên cây nhị phân

Bài 7: Cài đặt cây nhị phân sử dụng con trỏ có dữ liệu chứa kiểu số nguyên. Viết hàm kiểm tra xem một cây có phải là cân bằng hay không. Viết hàm tính tổng các số trên cây nhị phân.